

Sprint 6 Notes: Semantic Page Structure and Heading Hierarchy

Sprint 6 Goal

The goal of Sprint 6 is to build HTML pages that have clear meaning, not just boxes.

Before this sprint, you learned how to write individual HTML elements, full HTML boilerplates, text elements, links, paths, navigation, images, audio, video, SVG, and iframe content.

In Sprint 6, you learn how to arrange page content into a meaningful structure.

By the end of this sprint, you should be able to:

- Explain what semantic HTML means.
- Explain why semantic HTML is better than using `div` for everything.
- Use `header`, `nav`, `main`, `footer`, `section`, `article`, `aside`, and `figure` correctly.
- Use `div` only when no semantic element fits.
- Build a correct heading hierarchy using `h1` to `h6`.
- Use one main `h1` for a normal page.
- Avoid skipped heading levels.
- Avoid using headings only because of their visual size.
- Avoid old presentational elements such as `font`, `center`, and `big`.
- Debug common beginner mistakes in semantic page structure.

1. Where Sprint 6 Fits in Your HTML Progress

Sprints 1 to 5 taught you the basic parts of HTML.

Sprint 1

You learned elements, tags, attributes, void elements, `id`, `class`, and basic `img`.

Example:

```
<h1>My Page</h1>
<p>This is a paragraph.</p>

```

Sprint 2

You learned the full HTML document structure.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Page</title>
</head>
<body>
  <h1>My Page</h1>
</body>
</html>
```

Sprint 3

You learned how to mark text based on meaning.

Example:

```
<p><strong>Warning:</strong> Do not share your password.</p>
<p>The term <i>semantic HTML</i> means HTML with meaning.</p>
```

Sprint 4

You learned links, paths, contact links, and navigation.

Example:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="contact.html">Contact</a>
</nav>
```

Sprint 5

You learned media and embedded content.

Example:

```

```

Sprint 6

Now you combine those skills into a complete page layout.

Simple idea:

Sprints 1 to 5 taught you HTML pieces. Sprint 6 teaches you how to organize those pieces into a meaningful page.

2. What Semantic HTML Means

Semantic HTML means using HTML elements that describe the meaning of the content.

A semantic element tells the browser, assistive technology, search engines, and developers what the content is.

Example:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
</nav>
```

This means:

This group of links is navigation.

Bad semantic structure:

```
<div>
  <div>Home About Contact</div>
</div>
```

This might display on the page, but it does not clearly explain what the content means.

Better semantic structure:

```
<header>
  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>
```

This is better because:

- `header` says this is introductory page content.
- `nav` says this is navigation.
- `a` says each item is a link.

Simple rule:

Semantic HTML describes meaning. Non-semantic HTML only creates generic boxes.

3. Semantic HTML vs Non-Semantic HTML

Semantic HTML

Semantic HTML gives meaning.

Examples:

```
<header></header>
<nav></nav>
<main></main>
<section></section>
<article></article>
<aside></aside>
<footer></footer>
<figure></figure>
```

These elements tell us what kind of content they contain.

Non-semantic HTML

Non-semantic HTML does not explain meaning by itself.

Examples:

```
<div></div>  
<span></span>
```

A `div` only means:

This is a generic block container.

A `span` only means:

This is a generic inline container.

They are not wrong, but they should not replace semantic elements when a better element exists.

4. Why Semantic HTML Matters

Semantic HTML matters for three main reasons.

4.1 It helps assistive technology

Screen readers and other assistive tools can use semantic structure to help users move through a page.

For example, a screen reader can recognize:

- where the navigation is
- where the main content starts
- where the footer is
- what the headings are
- what content belongs to a section

This is much better than forcing the user to move through many meaningless `div` elements.

4.2 It can help SEO

Search engines read page structure to understand content.

Clear structure can help search engines understand:

- the main topic of the page
- the main sections of the page
- important navigation links
- article content
- related content

Semantic HTML does not magically guarantee high ranking, but it gives search engines cleaner structure.

4.3 It helps developers

Semantic HTML makes code easier to read.

Weak structure:

```
<div class="top">
  <div class="links">
    <a href="index.html">Home</a>
  </div>
</div>
```

Better structure:

```
<header>
  <nav>
    <a href="index.html">Home</a>
  </nav>
</header>
```

The second version is easier to understand.

A developer can immediately see:

- this is the header
- this is the navigation
- this is a link

Simple rule:

Semantic HTML makes your code clearer for browsers, tools, search engines, and humans.

5. The `header` Element

What `header` means

The `header` element represents introductory content.

It is often used at the top of a page or at the top of a section.

A page header often contains:

- site name
- page title
- logo
- short introduction
- navigation

Example:

```
<header>
  <h1>Code Learning Hub</h1>
  <nav>
    <a href="index.html">Home</a>
    <a href="courses.html">Courses</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>
```

This means:

This is the top introductory area of the page.

Common beginner mistake

Weak:

```
<div id="header">
  <h1>My Site</h1>
</div>
```

Better:

```
<header>
  <h1>My Site</h1>
</header>
```

Why better?

Because `header` already means page or section introduction. You do not need a generic `div` when a meaningful element exists.

6. The `nav` Element

What `nav` means

The `nav` element represents a navigation area.

Use it for important groups of links that help users move around the site or page.

Example:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="contact.html">Contact</a>
</nav>
```

This means:

These links are main navigation links.

When to use `nav`

Use `nav` for:

- main site menu
- page navigation
- important section navigation
- table of contents links

When not to use `nav`

Do not use `nav` for every single link.

Example:

```
<p>
  Read our <a href="privacy.html">privacy policy</a>.
</p>
```

This link does not need a `nav` wrapper because it is just a normal link inside a paragraph.

Simple rule:

Use `nav` for important navigation groups, not every link.

7. The `main` Element

What `main` means

The `main` element contains the main content of the page.

It should hold the unique central content of the page.

Example:

```
<main>
  <h2>Welcome</h2>
  <p>This page teaches beginner HTML structure.</p>
</main>
```

This means:

This is the main content of this page.

Important rule

A normal page should usually have one `main` element.

Do not use `main` for repeated content that appears on every page, such as:

- site header
- main navigation
- footer copyright

Those parts usually go outside `main`.

Common page structure:

```
<body>
  <header>
    <h1>My Site</h1>
  </header>

  <main>
    <h2>Page Content</h2>
```

```
<p>This is the main page content.</p>
</main>

<footer>
  <p>&copy; 2026 My Site</p>
</footer>
</body>
```

Simple rule:

`main` contains the central content of the page.

8. The `section` Element

What `section` means

The `section` element groups related content.

Use `section` when a part of the page has its own topic.

Example:

```
<section>
  <h2>Featured Courses</h2>
  <p>Choose a course to start learning.</p>
</section>
```

This means:

This is a section about featured courses.

A section should usually have a heading

A `section` should usually be introduced by a heading.

Weak:

```
<section>
  <p>We teach HTML, CSS, and JavaScript.</p>
</section>
```

Better:

```
<section>
  <h2>Our Courses</h2>
  <p>We teach HTML, CSS, and JavaScript.</p>
</section>
```

The heading gives the section a clear topic.

Good uses for `section`

Use `section` for:

- About section
- Services section
- Course list section
- Contact section
- FAQ section
- Features section
- Review section

Simple rule:

Use `section` when the content is a meaningful group with its own topic.

9. The `article` Element

What `article` means

The `article` element represents self-contained content.

Self-contained means the content could make sense on its own.

Example:

```
<article>
  <h2>How HTML Works</h2>
  <p>HTML gives structure and meaning to web content.</p>
</article>
```

This article could be copied to another page and still make sense.

Good uses for `article`

Use `article` for:

- blog post
- news article
- product review
- movie review
- forum post
- comment
- course card
- lesson preview

Example:

```
<section>
  <h2>Latest Articles</h2>

  <article>
    <h3>Understanding Semantic HTML</h3>
    <p>Semantic HTML helps browsers and people understand page structure.</p>
  </article>

  <article>
    <h3>Why Headings Matter</h3>
    <p>Headings create a clear outline for your page.</p>
  </article>
</section>
```

Simple test for `article`

Ask:

Could this content make sense if it were copied somewhere else?

If yes, `article` may be correct.

If no, use `section` or another element.

10. `section` vs `article`

Beginners often confuse `section` and `article`.

Use `section` when the content is a page section

Example:

```
<section>
  <h2>Why Learn HTML?</h2>
  <p>HTML is the foundation of web pages.</p>
</section>
```

This is a section of the page.

Use `article` when the content can stand alone

Example:

```
<article>
  <h2>HTML Beginner Guide</h2>
  <p>This guide explains the basics of HTML.</p>
</article>
```

This could be a standalone article.

Simple comparison

Element	Simple Meaning	Use It For
<code>section</code>	A grouped part of a page	About, features, services, FAQ
<code>article</code>	Self-contained content	Blog post, review, news article, course card

Simple rule:

`section` is a page group. `article` is content that can stand alone.

11. The `aside` Element

What `aside` means

The `aside` element represents related or extra content.

It is not the main content. It supports the main content.

Example:

```
<aside>
  <h2>Related Links</h2>
  <a href="resources.html">HTML resources</a>
</aside>
```

This means:

This content is related, but it is not the main part of the page.

Good uses for `aside`

Use `aside` for:

- related links
- author bio
- sidebar content
- extra notes
- advertisements
- recommended articles
- supporting information

Example:

```
<main>
  <article>
    <h2>Learning HTML</h2>
    <p>HTML is used to structure web content.</p>
  </article>

  <aside>
    <h2>Related Topics</h2>
    <a href="css.html">Learn CSS next</a>
  </aside>
</main>
```

Simple rule:

Use `aside` for content that is related to the main content but not essential to it.

12. The footer Element

What footer means

The footer element represents footer content for a page or section.

A page footer usually appears near the bottom of the page.

Example:

```
<footer>
  <p>&copy; 2026 Code Learning Hub</p>
</footer>
```

Common footer content

A footer may contain:

- copyright text
- contact links
- legal links
- privacy policy link
- terms link
- secondary navigation
- author information

Example:

```
<footer>
  <p>&copy; 2026 Code Learning Hub</p>
  <nav>
    <a href="privacy.html">Privacy Policy</a>
    <a href="terms.html">Terms of Use</a>
  </nav>
</footer>
```

Simple rule:

Use footer for bottom information about the page, site, or section.

13. The `figure` Element

What `figure` means

The `figure` element represents self-contained media or illustrative content.

It is often used with:

- images
- diagrams
- charts
- illustrations
- code examples

Example:

```
<figure>
  
  <figcaption>A basic semantic HTML page layout.</figcaption>
</figure>
```

The `figcaption` element

The `figcaption` element gives a caption for the figure.

Example:

```
<figcaption>A basic semantic HTML page layout.</figcaption>
```

The caption explains the figure.

Good use of `figure`

```
<figure>
  
  <figcaption>Poster for The Space Journey.</figcaption>
</figure>
```

Simple rule:

Use `figure` when media is a self-contained part of the content.

14. The `div` Element Is Still Allowed

Sprint 6 does not ban `div`.

The `div` element is still useful when no semantic element fits.

What `div` means

`div` is a generic block container.

It does not give special meaning.

Example:

```
<div class="card">
  <h3>HTML Basics</h3>
  <p>Learn how web pages are structured.</p>
</div>
```

This can be acceptable if the `div` is only used for grouping or styling.

Bad use of `div`

Weak:

```
<div id="header">
  <div>Home About Contact</div>
</div>
```

Better:

```
<header>
  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
    <a href="contact.html">Contact</a>
```

```
</nav>  
</header>
```

When `div` is acceptable

Use `div` when:

- you need a generic wrapper for layout
- you need a styling hook
- no semantic element describes the content
- you are grouping items only for CSS

Simple rule:

Use semantic elements when meaning exists. Use `div` only when you need a generic container.

15. Presentational Elements to Avoid

Some old HTML elements were used for appearance.

Examples:

```
<font>  
<center>  
<big>
```

These are presentational elements.

They focus on how content looks, not what content means.

Modern HTML should avoid them.

Bad

```
<center>Copyright 2026</center>
```

Better

```
<footer>
  <p>&copy; 2026 My Site</p>
</footer>
```

If you want to center text visually, use CSS later.

Simple rule:

HTML should describe structure and meaning. CSS should control appearance.

16. Heading Hierarchy

Headings create the outline of your page.

They are not just for making text big or small.

HTML has six heading levels:

```
<h1>Main heading</h1>
<h2>Section heading</h2>
<h3>Subsection heading</h3>
<h4>Smaller subsection heading</h4>
<h5>Lower-level heading</h5>
<h6>Lowest-level heading</h6>
```

Simple meaning of heading levels

Heading	Meaning
<code>h1</code>	Main page topic
<code>h2</code>	Major section
<code>h3</code>	Subsection of an <code>h2</code>
<code>h4</code>	Subsection of an <code>h3</code>
<code>h5</code>	Subsection of an <code>h4</code>
<code>h6</code>	Subsection of an <code>h5</code>

Simple rule:

Headings should show content structure, not visual size.

17. Use One Main `h1` on a Normal Page

For this curriculum, a normal page should have one main `h1`.

The `h1` should describe the main topic of the page.

Example:

```
<h1>Movie Review: The Space Journey</h1>
```

This tells the user:

This page is mainly about a movie review.

Then use `h2` for major sections.

Example:

```
<h1>Movie Review: The Space Journey</h1>

<h2>Summary</h2>
<h2>Review</h2>
<h2>Cast</h2>
<h2>Related Movies</h2>
```

Use `h3` for subsections inside an `h2` section.

Example:

```
<h2>Review</h2>
<h3>Story</h3>
<h3>Visuals</h3>
<h3>Music</h3>
```

Simple rule:

Use `h1` for the page topic. Use lower headings for sections and subsections.

18. Do Not Skip Heading Levels

Do not jump from `h1` directly to `h3`.

Bad:

```
<h1>HTML Course</h1>
<h3>HTML Elements</h3>
```

Why this is bad:

- The page skipped `h2`.
- The structure is unclear.
- Screen reader navigation can become confusing.
- The outline looks broken.

Better:

```
<h1>HTML Course</h1>
<h2>Beginner Lessons</h2>
<h3>HTML Elements</h3>
```

This creates a clear structure:

```
HTML Course
  Beginner Lessons
    HTML Elements
```

Simple rule:

Do not skip heading levels. Move down one level at a time.

19. Correct Heading Order Example

Good:

```
<h1>HTML Course</h1>

<h2>Beginner Lessons</h2>
<h3>HTML Elements</h3>
```

```
<h3>HTML Attributes</h3>

<h2>Practice Projects</h2>
<h3>Profile Page</h3>
<h3>Media Gallery</h3>
```

This outline makes sense:

```
HTML Course
  Beginner Lessons
    HTML Elements
    HTML Attributes
  Practice Projects
    Profile Page
    Media Gallery
```

The structure is clear because:

- `h1` is the page topic.
- `h2` elements are major sections.
- `h3` elements are subsections.

20. Bad Heading Order Example

Bad:

```
<h1>HTML Course</h1>
<h3>HTML Elements</h3>
<h2>Beginner Lessons</h2>
```

This is confusing.

It is like writing:

```
Chapter 1
  Sub-subsection
    Section
```

The section order does not make sense.

Fixed:

```
<h1>HTML Course</h1>
<h2>Beginner Lessons</h2>
<h3>HTML Elements</h3>
```

21. Do Not Use Headings Only for Visual Size

A common beginner mistake is choosing headings because of how they look.

Bad thinking:

I want large text, so I will use `h1`.

Better thinking:

This is the main page topic, so I will use `h1`.

Bad:

```
<h1>My Website</h1>
<h1>Huge Sale</h1>
<h1>Products</h1>
```

Problem:

The page has too many main headings. The structure is unclear.

Better:

```
<h1>My Website</h1>

<section>
  <h2>Huge Sale</h2>
  <p>Find our current offers below.</p>
</section>

<section>
  <h2>Products</h2>
  <p>Browse our featured products.</p>
</section>
```

If you want `h2` to look bigger or smaller, use CSS later.

Simple rule:

Use headings for structure. Use CSS for visual size.

22. Do Not Put Headings Inside Paragraphs

A heading should not be placed inside a paragraph.

Bad:

```
<p><h2>About Us</h2></p>
```

Why this is bad:

- A paragraph is for paragraph text.
- A heading is its own block of structure.
- This creates invalid or confusing HTML.

Better:

```
<h2>About Us</h2>
<p>We teach beginner HTML.</p>
```

Simple rule:

Headings and paragraphs should be separate elements.

23. Complete Semantic Page Skeleton

This is a common Sprint 6 page structure.

```
<body>
  <header>
    <h1>Site Name</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="about.html">About</a>
    </nav>
  </header>
```

```
<main>
  <section>
    <h2>Featured Articles</h2>

    <article>
      <h3>Understanding HTML</h3>
      <p>HTML gives the page structure.</p>
    </article>
  </section>

  <aside>
    <h2>Related Links</h2>
    <a href="resources.html">HTML resources</a>
  </aside>
</main>

<footer>
  <p>&copy; 2026 Example Site</p>
</footer>
</body>
```

This page uses:

- `header` for the top area
- `nav` for navigation
- `main` for central page content
- `section` for grouped content
- `article` for self-contained content
- `aside` for related links
- `footer` for bottom information
- correct heading order

24. Full Sprint 6 Example Page

Create a file named:

```
sprint-6-semantic-homepage.html
```

Example code:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Semantic Homepage</title>
<meta name="description" content="A beginner semantic HTML homepage.">
</head>
<body>
  <header>
    <h1>Code Learning Hub</h1>

    <nav>
      <a href="index.html">Home</a>
      <a href="courses.html">Courses</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Featured Courses</h2>

      <article>
        <h3>HTML Fundamentals</h3>
        <p>Learn how to structure web pages with meaningful HTML.</p>
      </article>
    </section>

    <section>
      <h2>Why Learn With Us?</h2>
      <p>Our lessons focus on clear structure, accessible markup, and beginner-
friendly practice.</p>
    </section>

    <aside>
      <h2>Related Links</h2>
      <a href="resources.html">View HTML learning resources</a>
    </aside>
  </main>

  <footer>
    <p>&copy; 2026 Code Learning Hub</p>
  </footer>
</body>
</html>
```

25. Explanation of the Full Example

Boilerplate

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Semantic Homepage</title>
</head>
<body>
  ...
</body>
</html>
```

This creates a complete HTML document.

Header

```
<header>
  <h1>Code Learning Hub</h1>
  ...
</header>
```

The `header` introduces the page.

The `h1` gives the main page topic.

Navigation

```
<nav>
  <a href="index.html">Home</a>
  <a href="courses.html">Courses</a>
  <a href="contact.html">Contact</a>
</nav>
```

The `nav` groups important navigation links.

Each link uses `a` and `href`.

Main content

```
<main>
  ...
</main>
```

The `main` contains the central content of the page.

First section

```
<section>
  <h2>Featured Courses</h2>
  ...
</section>
```

This is a major section of the page.

The `h2` gives the section topic.

Article

```
<article>
  <h3>HTML Fundamentals</h3>
  <p>Learn how to structure web pages with meaningful HTML.</p>
</article>
```

This is a self-contained course card or lesson preview.

It uses `h3` because it is inside the `h2` section.

Second section

```
<section>
  <h2>Why Learn With Us?</h2>
  <p>Our lessons focus on clear structure, accessible markup, and beginner-
friendly practice.</p>
</section>
```

This is another major section of the page.

Aside

```
<aside>
  <h2>Related Links</h2>
  <a href="resources.html">View HTML learning resources</a>
</aside>
```

The `aside` contains related supporting content.

Footer

```
<footer>
  <p>&copy; 2026 Code Learning Hub</p>
</footer>
```

The `footer` contains bottom page information.

26. Page Outline of the Full Example

The page structure looks like this:

```
body
  header
    h1
    nav
  main
    section
      h2
      article
        h3
    section
      h2
    aside
      h2
  footer
```

The heading outline looks like this:

```
Code Learning Hub
  Featured Courses
```

This is clear because:

- the page has one main `h1`
- major sections use `h2`
- the article inside a section uses `h3`
- no heading levels are skipped

27. Movie Review Example

This example uses `header`, `nav`, `main`, `article`, `section`, `figure`, `aside`, and `footer`.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Movie Review Page</title>
  <meta name="description" content="A beginner semantic HTML movie review
page.">
</head>
<body>
  <header>
    <h1>Movie Review: The Space Journey</h1>

    <nav>
      <a href="index.html">Home</a>
      <a href="reviews.html">Movie Reviews</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>Review Overview</h2>
      <p>
        The Space Journey is a science fiction movie about exploration,
        teamwork, and survival.
      </p>

      <section>
```

```

        <h3>Story</h3>
        <p>
            The story follows a small crew traveling beyond Earth to find a new
home.
        </p>
    </section>

    <section>
        <h3>Visuals</h3>
        <p>
            The movie uses strong visual scenes to show planets, ships, and space
travel.
        </p>
    </section>

    <figure>
        
        <figcaption>Poster for The Space Journey.</figcaption>
    </figure>
</article>

<aside>
    <h2>Related Reviews</h2>
    <a href="moon-review.html">Read our Moon Mission review</a>
</aside>
</main>

<footer>
    <p>&copy; 2026 Movie Review Site</p>
</footer>
</body>
</html>

```

Structure of the movie review page

```

body
  header
    h1
    nav
  main
    article
      h2
      section
        h3
      section

```

```
h3
figure
aside
h2
footer
```

This is a real semantic page structure.

It is not just loose boxes.

28. Debugging Sprint 6 Broken Code

Broken code:

```
<div id="header">
  <div>Home About Contact</div>
</div>
<div id="main">
  <h1>My Site</h1>
  <p><h1>Huge Sale</h1></p>
  <h3>Products</h3>
</div>
<center>Copyright</center>
```

Problems

Problem 1: Generic `div` used for the header

Broken:

```
<div id="header">
```

Better:

```
<header>
```

Why?

Because the content is a header. Use the semantic element.

Problem 2: Navigation is just plain text

Broken:

```
<div>Home About Contact</div>
```

Better:

```
<nav>  
  <a href="index.html">Home</a>  
  <a href="about.html">About</a>  
  <a href="contact.html">Contact</a>  
</nav>
```

Why?

Navigation should be real links inside `nav`.

Problem 3: Generic `div` used for main content

Broken:

```
<div id="main">
```

Better:

```
<main>
```

Why?

The content is the main page content.

Problem 4: Heading inside paragraph

Broken:

```
<p><h1>Huge Sale</h1></p>
```

Better:

```
<h2>Huge Sale</h2>
<p>Find our current product offers below.</p>
```

Why?

A heading should not be placed inside a paragraph.

Problem 5: Too many `h1` elements

Broken:

```
<h1>My Site</h1>
<h1>Huge Sale</h1>
```

Better:

```
<h1>My Site</h1>
<h2>Huge Sale</h2>
```

Why?

Use one main `h1` for the page topic. Use `h2` for major sections.

Problem 6: Skipped heading level

Broken:

```
<h1>My Site</h1>
<h3>Products</h3>
```

Better:

```
<h1>My Site</h1>
<h2>Products</h2>
```

Why?

Do not skip from `h1` to `h3`.

Problem 7: Deprecated `center` element

Broken:

```
<center>Copyright</center>
```

Better:

```
<footer>
  <p>&copy; 2026 My Site</p>
</footer>
```

Why?

`center` is presentational. Use semantic HTML for structure and CSS later for centering.

Fixed code

```
<header>
  <h1>My Site</h1>

  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>

<main>
  <section>
    <h2>Huge Sale</h2>
    <p>Find our current product offers below.</p>
  </section>

  <section>
    <h2>Products</h2>
    <p>Browse our featured products.</p>
  </section>
</main>

<footer>
  <p>&copy; 2026 My Site</p>
</footer>
```

This fixed version is better because:

- it uses `header`
 - it uses `nav`
 - it uses real links
 - it uses `main`
 - headings are not inside paragraphs
 - the page has one main `h1`
 - heading levels are not skipped
 - it uses `footer`
 - it avoids `center`
-

29. Common Sprint 6 Mistakes and Fixes

Mistake 1: Building the whole page with `div`

Bad:

```
<div id="header"></div>
<div id="main"></div>
<div id="footer"></div>
```

Better:

```
<header></header>
<main></main>
<footer></footer>
```

Mistake 2: Using `div` instead of `nav`

Bad:

```
<div class="navigation">
  <a href="index.html">Home</a>
</div>
```

Better:

```
<nav>
  <a href="index.html">Home</a>
</nav>
```

Mistake 3: Skipping heading levels

Bad:

```
<h1>Page Title</h1>
<h3>Section Title</h3>
```

Better:

```
<h1>Page Title</h1>
<h2>Section Title</h2>
```

Mistake 4: Using headings for size

Bad:

```
<h1>My Site</h1>
<h1>Products</h1>
```

Better:

```
<h1>My Site</h1>
<h2>Products</h2>
```

Mistake 5: Putting a heading inside a paragraph

Bad:

```
<p><h2>About Us</h2></p>
```

Better:

```
<h2>About Us</h2>
<p>We teach beginner HTML.</p>
```

Mistake 6: Using `center`

Bad:

```
<center>Copyright 2026</center>
```

Better:

```
<footer>
  <p>&copy; 2026 Example Site</p>
</footer>
```

Mistake 7: Using a section without a clear topic

Weak:

```
<section>
  <p>Random content here.</p>
</section>
```

Better:

```
<section>
  <h2>About This Page</h2>
  <p>This page explains semantic HTML.</p>
</section>
```

Mistake 8: Using `article` for content that cannot stand alone

Weak:

```
<article>
  <h2>Contact Form</h2>
</article>
```

This may not be an article unless it is self-contained content.

Better if it is just a page section:

```
<section>
  <h2>Contact Us</h2>
</section>
```

30. Warm-Up Drills

Practice these before building a full page.

Drill 1: Write a header with nav

```
<header>
  <h1>My Website</h1>
  <nav>
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>
```

Drill 2: Write a main element with two sections

```
<main>
  <section>
    <h2>About</h2>
    <p>This page teaches semantic HTML.</p>
  </section>

  <section>
    <h2>Lessons</h2>
    <p>Start with HTML structure and headings.</p>
  </section>
</main>
```

Drill 3: Write a footer

```
<footer>
  <p>&copy; 2026 My Website</p>
</footer>
```

Drill 4: Write a correct `h1`, `h2`, `h3` outline

```
<h1>HTML Course</h1>
<h2>Beginner Lessons</h2>
<h3>Semantic HTML</h3>
```

Drill 5: Write an article inside a section

```
<section>
  <h2>Latest Lessons</h2>

  <article>
    <h3>HTML Structure</h3>
    <p>Learn how to organize page content.</p>
  </article>
</section>
```

31. Guided Coding Task

Create a semantic homepage.

Requirements

Your page should include:

1. Full Sprint 2 boilerplate.
2. `header`.
3. One `h1` inside the header.
4. `nav` with at least three links.
5. `main`.
6. Two `section` elements inside `main`.
7. Each section should have an `h2`.
8. One `article` with an `h3` inside one section.
9. One `aside` with related links.

10. One `footer`.
11. No skipped heading levels.
12. No `center`, `font`, or `big`.
13. Use `div` only if no semantic element fits.

Starter solution

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Semantic Homepage</title>
  <meta name="description" content="A beginner semantic homepage built with
HTML.">
</head>
<body>
  <header>
    <h1>My Learning Website</h1>

    <nav>
      <a href="index.html">Home</a>
      <a href="lessons.html">Lessons</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Featured Lessons</h2>

      <article>
        <h3>Semantic HTML</h3>
        <p>Learn how to use meaningful HTML elements for page structure.</p>
      </article>
    </section>

    <section>
      <h2>Why Structure Matters</h2>
      <p>Good structure helps users, search engines, and developers understand
your page.</p>
    </section>

    <aside>
      <h2>Related Resources</h2>
      <a href="resources.html">View beginner HTML resources</a>
    </aside>
```

```
</main>

<footer>
  <p>&copy; 2026 My Learning Website</p>
</footer>
</body>
</html>
```

32. Build Something Small: Movie Review Page

Build a semantic movie review page.

Requirements

Your page should include:

- full HTML boilerplate
- header
- nav
- main
- article
- at least two section elements
- aside
- footer
- one figure
- one image with alt
- one figcaption
- one main h1
- no skipped heading levels

Starter idea

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Movie Review</title>
</head>
<body>
  <header>
    <h1>Movie Review: Ocean Planet</h1>
    <nav>
```

```

    <a href="index.html">Home</a>
    <a href="reviews.html">Reviews</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>

<main>
  <article>
    <h2>Review Summary</h2>
    <p>Ocean Planet is a story about discovery and survival.</p>

    <section>
      <h3>Story</h3>
      <p>The story follows a crew exploring a mysterious ocean world.</p>
    </section>

    <section>
      <h3>Visuals</h3>
      <p>The movie uses underwater scenes and wide landscapes.</p>
    </section>

    <figure>
      
      <figcaption>Poster for Ocean Planet.</figcaption>
    </figure>
  </article>

  <aside>
    <h2>Related Reviews</h2>
    <a href="space-review.html">Read our Space Journey review</a>
  </aside>
</main>

<footer>
  <p>&copy; 2026 Movie Review Site</p>
</footer>
</body>
</html>

```

33. Challenge Task

Add a `figure` element to your semantic page.

Your `figure` should contain:

- one `img`
- useful `alt` text
- one `figcaption`

Example:

```
<figure>
  
  <figcaption>Basic semantic HTML page regions.</figcaption>
</figure>
```

Make sure the image is part of the content, not just decoration.

34. Sprint 6 Revision Questions

Use these questions to test yourself.

Question 1

Why is `nav` better than `div` for a navigation section?

Answer:

`nav` clearly means navigation. A `div` is only a generic container and does not explain meaning.

Question 2

When is `div` acceptable?

Answer:

Use `div` when no semantic element fits and you only need a generic container for grouping or styling.

Question 3

How many `h1` elements should a normal page have in this curriculum?

Answer:

A normal page should have one main `h1` for the page topic.

Question 4

Why is `h1` followed directly by `h3` a problem?

Answer:

It skips `h2`, which breaks the heading structure and makes the page outline confusing.

Question 5

Name three benefits of semantic HTML.

Answer:

Semantic HTML helps assistive technology, search engines, and developers.

Question 6

Which deprecated presentational elements should you avoid?

Answer:

Avoid `font`, `center`, and `big`.

Question 7

What element would you use for a self-contained blog post?

Answer:

Use `article`.

Question 8

What element would you use for related side content?

Answer:

Use `aside`.

Question 9

What element contains the main page content?

Answer:

Use `main`.

Question 10

What element usually holds bottom page information?

Answer:

Use `footer`.

Question 11

What element groups related content with a topic?

Answer:

Use `section`.

Question 12

What element should you use for a self-contained image with a caption?

Answer:

Use `figure` with `figcaption`.

Question 13

Should you use headings only because they look large?

Answer:

No. Use headings for structure. Use CSS later for visual size.

Question 14

Should a heading be placed inside a paragraph?

Answer:

No. Headings and paragraphs should be separate elements.

35. Sprint 6 Key Rules

Memorize these rules:

- Semantic HTML means HTML with meaning.
 - Use `header` for introductory page or section content.
 - Use `nav` for important navigation links.
 - Use `main` for the central content of the page.
 - Use `section` for grouped related content.
 - Use `article` for self-contained content.
 - Use `aside` for related side content.
 - Use `footer` for bottom page or section information.
 - Use `figure` for self-contained media or examples.
 - Use `figcaption` to caption a figure.
 - Do not use `div` for everything.
 - Use `div` only when no semantic element fits.
 - Use one main `h1` for a normal page.
 - Use `h2` for major sections.
 - Use `h3` for subsections of `h2`.
 - Do not skip heading levels.
 - Do not use headings only for visual size.
 - Do not put headings inside paragraphs.
 - Avoid old presentational elements like `font`, `center`, and `big`.
 - Use CSS later for appearance.
-

36. Sprint 6 Mini Glossary

Semantic HTML

HTML that describes the meaning of content.

Example:

```
<nav>...</nav>
```

Non-semantic HTML

HTML that does not describe meaning by itself.

Example:

```
<div>...</div>
```

header

Introductory content for a page or section.

nav

A section of important navigation links.

main

The main content of the page.

section

A grouped part of the page with a topic.

article

Self-contained content that can stand alone.

aside

Related or supporting content.

footer

Bottom information for a page or section.

figure

Self-contained media, diagram, image, or code example.

figcaption

A caption for a `figure`.

div

A generic block container with no special meaning.

Heading hierarchy

The structure created by `h1` to `h6` headings.

h1

The main page topic.

h2

A major section.

h3

A subsection of an `h2`.

Presentational element

An old-style element that focuses on appearance instead of meaning.

Examples:

```
<font>
<center>
<big>
```

37. Final Self-Check

You are ready to move on from Sprint 6 when you can do all of these without looking at notes:

- Explain what semantic HTML means.
- Explain why `nav` is better than `div` for navigation.

- Build a page with `header`, `nav`, `main`, and `footer`.
 - Use `section` for grouped content.
 - Use `article` for self-contained content.
 - Use `aside` for related content.
 - Use `figure` and `figcaption` correctly.
 - Explain when `div` is still acceptable.
 - Use one main `h1`.
 - Use `h2` and `h3` in correct order.
 - Fix skipped heading levels.
 - Avoid putting headings inside paragraphs.
 - Avoid `font`, `center`, and `big`.
 - Explain why CSS should control appearance.
 - Debug broken semantic HTML.
-

38. Final Mental Model

When writing HTML, ask this first:

What is this content?

Then choose the element.

If it is page introduction, use `header`.

If it is navigation, use `nav`.

If it is main content, use `main`.

If it is a topic group, use `section`.

If it can stand alone, use `article`.

If it is related side content, use `aside`.

If it is bottom information, use `footer`.

If it is self-contained media with a caption, use `figure` and `figcaption`.

If no semantic element fits, use `div`.

Most important Sprint 6 rule:

Think about meaning before appearance.